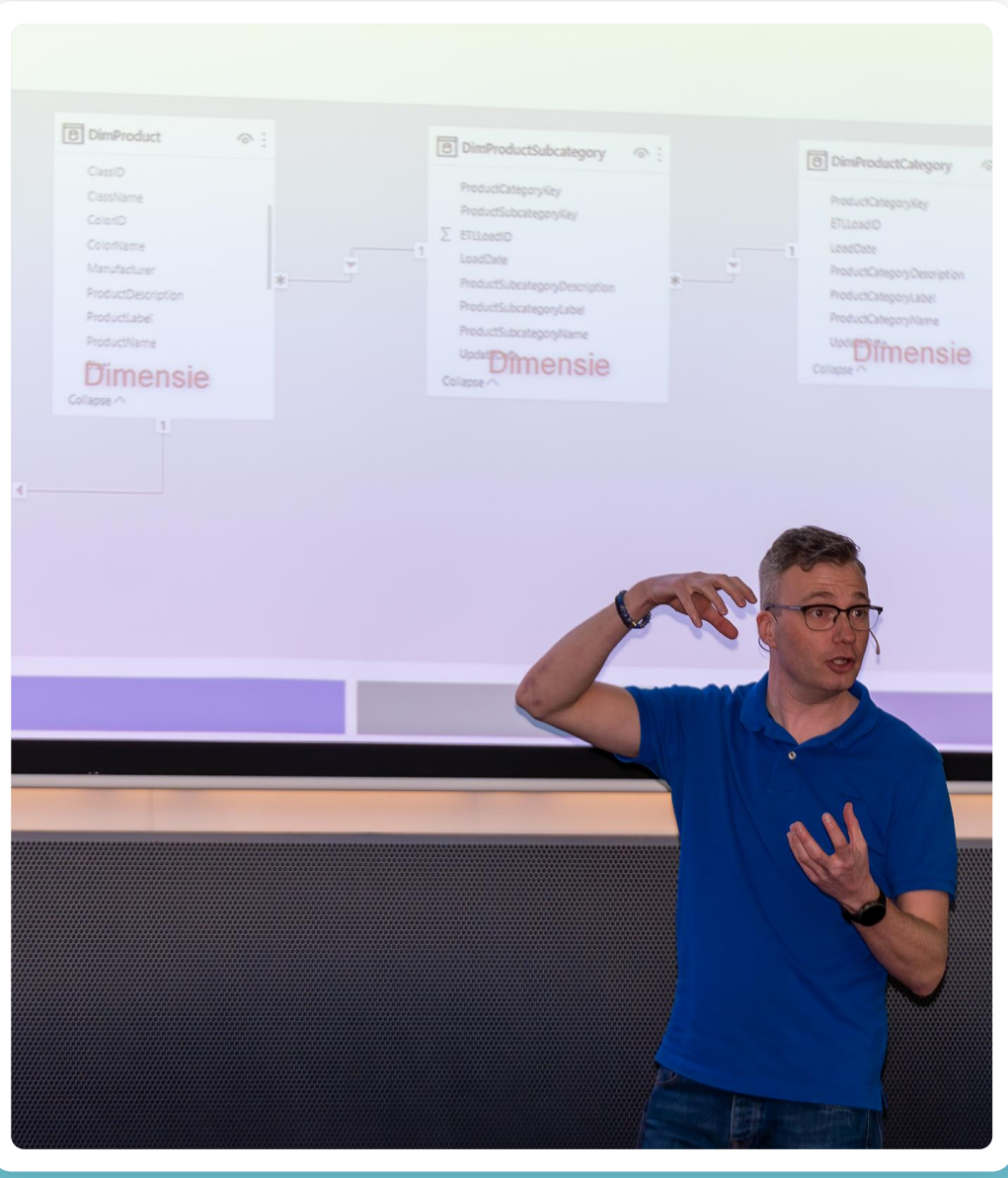
dutchBIgeek

Power BI Expert Training



Peter van den Bos

Data Modeling & DAX Specialist
Freelance Trainer & Speaker
Founder @ DutchBiGeek

dutchbigeek.nl
linkedin.com/in/petervdbos
github.com/PetervandenBos/
sessionize.com/peter-van-den-bos
youtube.com/@DutchBIgeek
peter@dutchbigeek.nl



🧠 **Geek by design**

⚡ *Because performance isn't optional*

Praktische zaken

Opening

09:00

Start

Einde om 16:00

15 min

Korte pauzes

Halverwege de ochtend en de middag

45 min

Lunch

Van 12:00 tot 12:45

Meten

Trainingsstijl

Voorspellen, meten, herschrijven

Nodig: Power BI Desktop, DAX Studio en het oefenmodel Trage-Contoso. Telefoon op stil, vragen mogen altijd.

Dagprogramma

Opening

09:00	●	Opening en nulmeting
09:20	1	Korte herhaling
10:35	2	VertiPaq en het datamodel
12:45	3	Engine en materialisatie
14:15	4	DAX Studio en optimaliseren
15:15	★	Praktijkcase: een traag rapport
15:50	●	Nameting en afsluiting
		Lunch 12:00

Zo werken we vandaag

Opening

1 → 2 → 3 → 4

Probleem

Een measure die fout of traag is

Voorspellen

Wat komt eruit, en hoeveel SE-queries?

Meten

Server Timings en queryplan

Oplossen

Herschrijven en opnieuw meten

Oefenmodel: Trage-Contoso

Sales (2 miljoen rijen), Product, Customer, Date en Store. Basismeasure: Sales Amount = SUMX (Sales, Sales[Quantity] * Sales[Net Price])

Nulmeting: drie voorspellingen

Opening

A Rij Blue?

```
-- matrix: Product[Color]
CALCULATE (
  [Sales Amount],
  KEEPFILTERS (
    Product[Color] = "Red" ),
  ALL ( Product[Color] )
)
```

B Aantal SE-queries?

```
EVALUATE
SUMMARIZECOLUMNS (
  Product[Brand],
  "Omzet", [Sales Amount],
  "Aantal", [Total Quantity],
  "Rood", [Sales Red],
  "Blauw", [Sales Blue]
)
```

C Welke is sneller?

```
-- 1
SUMX ( Sales,
  IF ( Sales[Unit Price]
         >= 500,
    Sales[Quantity]
    * Sales[Net Price] ) )

-- 2
CALCULATE ( [Sales Amount],
  KEEPFILTERS (
    Sales[Unit Price]
      >= 500 ) )
```

dutchBIgeek

Korte herhaling

“Je kent DAX. Maar weet je ook wat het kost?”

BLOK 1

In dit blok

- Context transition
- Iterators
- Volgorde van CALCULATE
- Column of table filter

Elke measure is een CALCULATE

Blok 1

```
Customer[Omzet klant] =
    [Sales Amount]

-- wordt door de engine gelezen als
CALCULATE ( [Sales Amount] )
```

Zie je een measure binnen een iterator, dan zie je CALCULATE.
En dus een context transition per rij.

Wat het kost

- Per rij een nieuwe filter context, op alle kolommen van die rij.
- Op een dimension van duizend rijen is dat niets. Op een fact table van miljoenen wel.
- Zijn rijen niet uniek, dan telt de context transition ze ook nog dubbel.

Iterators: kies de goedkoopste tabel

Blok 1

```
-- transitie per verkoopregel
Omzet A =
SUMX ( Sales, [Sales Amount] )
```

```
-- transitie per klant
Omzet B =
SUMX ( Customer, [Sales Amount] )

-- geen transitie
Omzet C = [Sales Amount]
```

Waar het om gaat

- Kosten volgen het aantal rijen dat je doorloopt, niet de lengte van de formule.
- Itereer over een dimension of VALUES van een kolom, niet over de fact table.
- Geneste iterators vermenigvuldigen het aantal rijen.
- In blok 4 meten we A, B en C.

De volgorde waarin CALCULATE werkt

Blok 1

1. Filter arguments berekenen in de oorspronkelijke context
2. Huidige filter context kopiëren
3. Context transition toepassen als er een row context is
4. Modifiers toepassen: ALL, REMOVEFILTERS, USERELATIONSHIP, CROSSFILTER
5. Filter arguments toepassen: vervangen, of combineren met KEEPFILTERS
6. Expressie berekenen in de nieuwe filter context

Nulmeting A: ALL haalt in stap 4 het filter op Blue weg, vóór KEEPFILTERS in stap 5.

Filter op een kolom, niet op een tabel

Blok 1

```
Rood, kolom =
CALCULATE (
    [Sales Amount],
    Product[Color] = "Red"
)
```

```
Rood, tabel =
CALCULATE (
    [Sales Amount],
    FILTER (
        Product,
        Product[Color] = "Red" ) )
```

Column filter

Een tabel met één kolom en een paar waarden. Klein, snel, en het vervangt alleen het filter op Color.

Table filter

Een tabel met alle kolommen van Product in de huidige context. Groter, en het werkt anders samen met bestaande filters.

Oefening blok 1

Blok 1

1

Rangschikken

Zet Omzet A, B en C op volgorde van snelheid en schrijf op waarom. We meten het in blok 4.

2

Volgorde voorspellen

Wat geeft CALCULATE ([Sales Amount], Product[Color] = "Red", REMOVEFILTERS (Product)) op de rij Blue?

3

Verborgen transitie

Zoek in het oefenmodel een measure die binnen een iterator over Sales een andere measure aanroept.

dutchBIgeek

VertiPaq en het datamodel

"Het model is 4 GB en elke visual is traag."

BLOK 2

In dit blok

- VertiPaq-opslag
- Encoding
- Segmenten
- Geheugengebruik
- Model verkleinen
- Relationships

Wat is VertiPaq?

Blok 2

- **In-Memory mode:**
  - DAX Formula Engine
  - VertiPaq Storage Engine
  - In-memory columnar database
  - Met alle Tabular opties

- **Direct Query-modus:**
  - DAX naar SQL
  - Optimalisatie vereist tuning van relationele database

VertiPaq in het kort

- Het is een in-memory database
- Gebaseerd op de relationele methodologie
- Kolom georiënteerde database

Opslag per rij

Blok 2

StoreKey	StoreName	ZipCode	StorePhone	AddressLine1
15	Contoso Renton Store	97001	424-555-0189	Renton,Citycenter
17	Contoso Spokane Store	97001	984-555-0185	Logan Ave Shopping mall
20	Contoso Englewood Store	87001	435-555-0113	Cherry Hills Village Shopping mall
22	Contoso Westminster Store	87001	925-555-0114	South Central Shopping mall
25	Contoso Greeley No.1 Store	80631	332-555-0150	W 29th Street, Greeley CityCenter
27	Contoso Lafayette Store	87001	113-555-0173	Lafayette Downtown
30	Contoso Fort Collins Store	87001	656-555-0119	Old Town West ST
32	Contoso Milliken Store	87001	523-555-0175	Milliken Downtown
35	Contoso Denver No.2 Store	87001	952-555-0178	West Highland Borden Ave
37	Contoso Boulder Store	87001	937-555-0137	Transit Village Broadway
40	Contoso Aurora Store	87001	794-555-0159	Willow Park Ave
42	Contoso Green Bay Store	54001	286-555-0189	Green Bay Downtown
45	Contoso Oshkosh Store	54001	786-555-0144	Oshkosh, WI Broadway
47	Contoso Milwaukee No.1 Store	54001	818-555-0192	Havenwoods St

Opslag per kolom

Blok 2

StoreKey	StoreName	ZipCode	StorePhone	AddressLine1
15	Contoso Renton Store	97001	424-555-0189	Renton,Citycenter
17	Contoso Spokane Store	97001	984-555-0185	Logan Ave Shopping mall
20	Contoso Englewood Store	87001	435-555-0113	Cherry Hills Village Shopping mall
22	Contoso Westminster Store	87001	925-555-0114	South Central Shopping mall
25	Contoso Greeley No.1 Store	80631	332-555-0150	W 29th Street, Greeley CityCenter
27	Contoso Lafayette Store	87001	113-555-0173	Lafayette Downtown
30	Contoso Fort Collins Store	87001	656-555-0119	Old Town West ST
32	Contoso Milliken Store	87001	523-555-0175	Milliken Downtown
35	Contoso Denver No.2 Store	87001	952-555-0178	West Highland Borden Ave
37	Contoso Boulder Store	87001	937-555-0137	Transit Village Broadway
40	Contoso Aurora Store	87001	794-555-0159	Willow Park Ave
42	Contoso Green Bay Store	54001	286-555-0189	Green Bay Downtown
45	Contoso Oshkosh Store	54001	786-555-0144	Oshkosh, WI Broadway
47	Contoso Milwaukee No.1 Store	54001	818-555-0192	Havenwoods St

Een query op kolomopslag

Blok 2

StoreKey	StoreName	ZipCode	StorePhone	AddressLine1
15	Contoso Renton Store	97001	424-555-0189	Renton,Citycenter
17	Contoso Spokane Store	97001	984-555-0185	Logan Ave Shopping mall
20	Contoso Englewood Store	87001	435-555-0113	Cherry Hills Village Shopping mall
22	Contoso Westminster Store	87001	925-555-0114	South Central Shopping mall
25	Contoso Greeley No.1 Store	80631	332-555-0150	W 29th Street, Greeley CityCenter
27	Contoso Lafayette Store	87001	113-555-0173	Lafayette Downtown
30	Contoso Fort Collins Store	87001	656-555-0119	Old Town West ST
32	Contoso Milliken Store	87001	523-555-0175	Milliken Downtown
35	Contoso Denver No.2 Store	87001	952-555-0178	West Highland Borden Ave
37	Contoso Boulder Store	87001	937-555-0137	Transit Village Broadway
40	Contoso Aurora Store	87001	794-555-0159	Willow Park Ave
42	Contoso Green Bay Store	54001	286-555-0189	Green Bay Downtown
45	Contoso Oshkosh Store	54001	786-555-0144	Oshkosh, WI Broadway
47	Contoso Milwaukee No.1 Store	54001	818-555-0192	Havenwoods St

Kolom- of rijopslag

- **Kolom:**
  - Snel toegang tot een enkele kolom
  - Tijd nodig om rijen te materialiseren
  - Trade CPU vs. I/O

- **Rij:**
  - Snel toegang tot een enkele rij
  - Geen materialisatie nodig
  - Trade I/O vs. CPU

VertiPaq-compressie

UnitCost	StoreName
€ 10,00	Contoso North America Online Store
€ 11,00	Contoso Europe Online Store
€ 14,00	Contoso Europe Online Store
€ 18,00	Contoso Europe Online Store
€ 15,00	Contoso North America Online Store
€ 14,00	Contoso North America Online Store
€ 19,00	Contoso Asia Online Store
€ 14,00	Contoso North America Online Store
€ 17,00	Contoso Europe Online Store
€ 19,00	Contoso Asia Online Store

Dictionary Encoding

Vervangt elke unieke waarde door een index in een dictionary, bij elk datatype

Bespaart geheugen: tekst → geheel getal

Waarden worden één keer opgeslagen in een woordenboek en via sleutels benaderd

Vooral effectief bij kolommen met een lage cardinaliteit

StoreName	StoreName.ID
Contoso North America Online Store	1
Contoso Europe Online Store	2
Contoso Europe Online Store	2
Contoso Europe Online Store	2
Contoso North America Online Store	1
Contoso North America Online Store	1
Contoso Asia Online Store	3
Contoso North America Online Store	1
Contoso Europe Online Store	2
Contoso Asia Online Store	3

Run-Length Encoding (RLE)

Blok 2

Comprimeert opeenvolgende herhalingen van dezelfde waarde

Slaat op als (waarde + aantal)

Werkt alleen goed wanneer de kolom gesorteerd is

Wordt alleen toegepast als het daadwerkelijk geheugen bespaart

StoreName.ID
1
2
2
2
1
1
3
1
2
3

StoreName
1
1
1
1
2
2
2
2
3
3

StoreName.ID	Count	Rows
1	0	4
2	4	4
3	7	2

Value Encoding

VertiPaq kiest het kleinste aantal bits per kolom

Kleinere waarde-interval = minder bits nodig

Alleen voor numerieke kolommen. Lukt het niet, dan kiest VertiPaq dictionary encoding

Original	Optimized	
UnitCost	UnitCost	
€ 10,00	0	+10
€ 11,00	1	+10
€ 14,00	4	+10
€ 18,00	8	+10
€ 15,00	5	+10
€ 14,00	4	+10
€ 19,00	9	+10
€ 14,00	4	+10
€ 17,00	7	+10
€ 19,00	9	+10

Segmenten

- **Elke tabel is verdeeld in segmenten:**
  - 8 miljoen rijen voor elk segment in SSAS
  - 1 miljoen rijen in Power Pivot en Power BI

  - Dictionary algemeen voor tabel
  - Bit-sizing
  - DMV beschikbaar om die info op te vragen

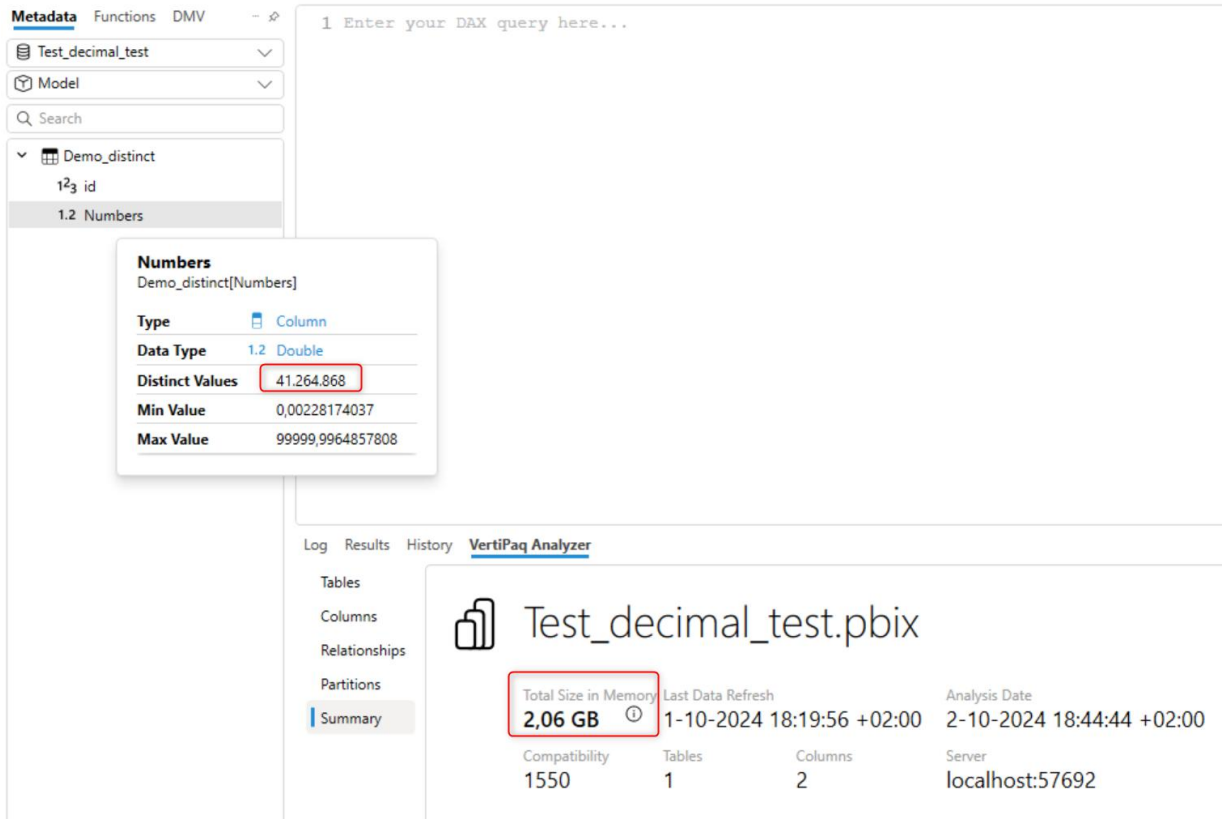
Geheugengebruik van de data

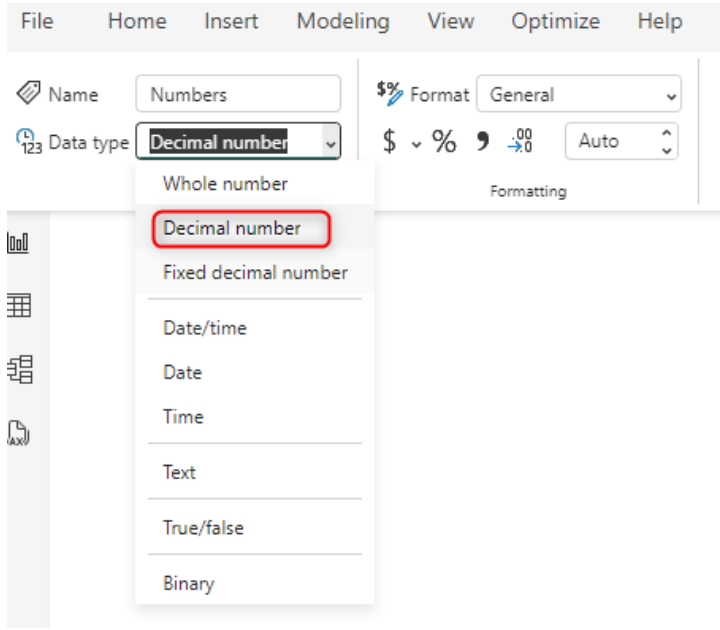
- **Het geheugengebruik hangt af van**:
  - Aantal kolommen
  - Cardinaliteit van elke kolom
  - Soort gegevens
  - Aantal rijen

- **Strings:**
  - Gemiddelde grootte is relevant voor grootte dictionary
  - Er is geen gemakkelijke formule

Voorbeeld decimaal getal

Blok 2

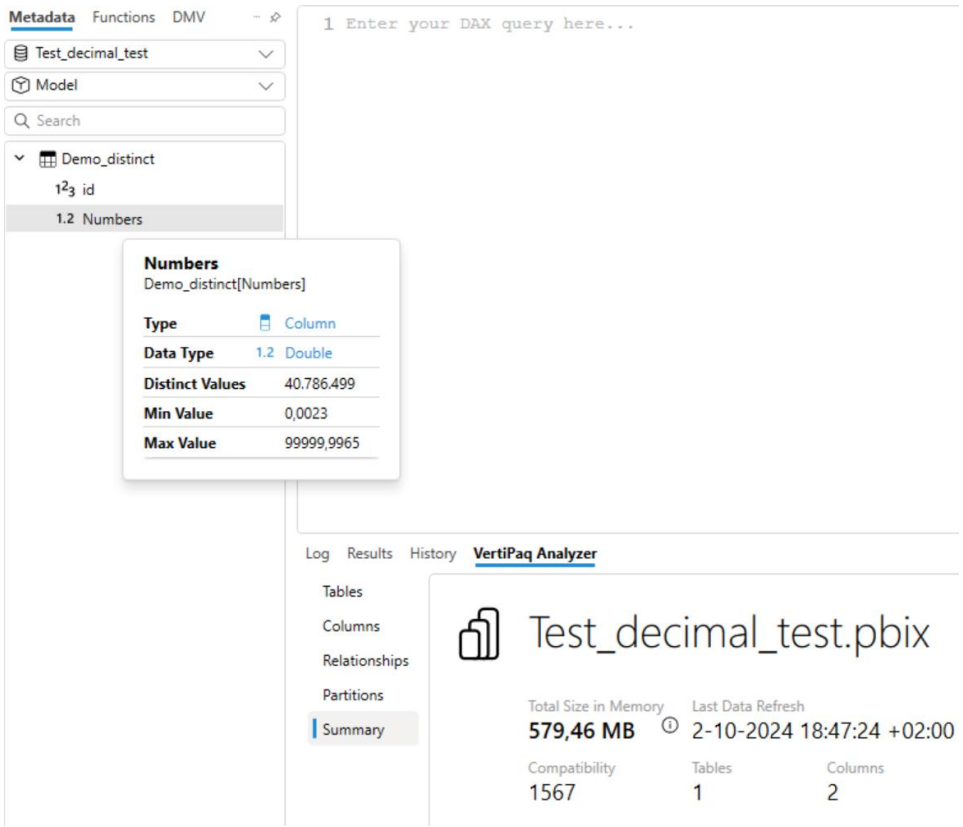


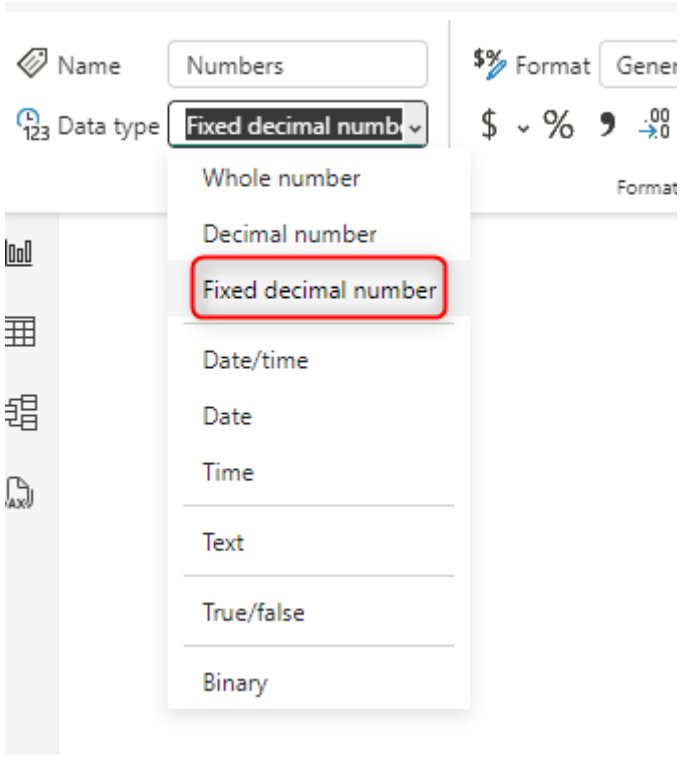


dutchBIgeek

Voorbeeld Fixed decimaal getal

Blok 2





dutchBIgeek

Dictionary verkleinen

Blok 2

- **Aantal verschillende waarden verminderen:**
  - Date Time: opgesplitst in twee kolommen
  - Date
  - Time
  - Decimaalwaarden: vaste precisie 10.231 > 10.2
- **Strings korter maken**
- **Dit moet allemaal gebeuren in brongegevens (bv. SQL-weergaven), niet in berekende kolommen**

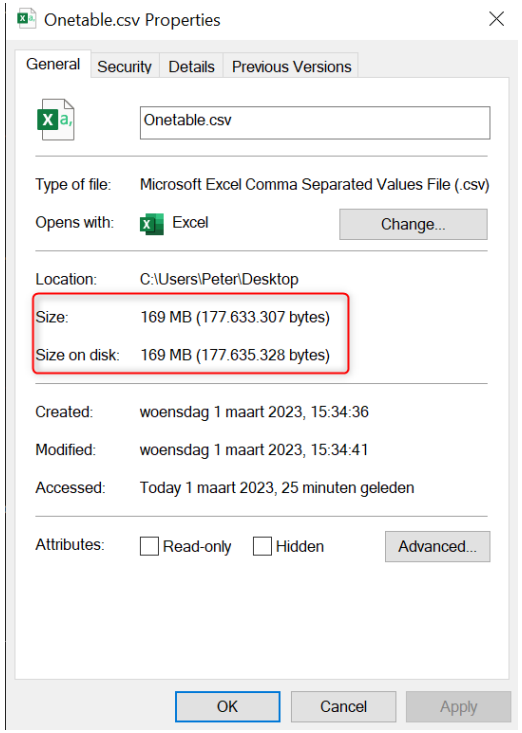
Tabel verkleinen

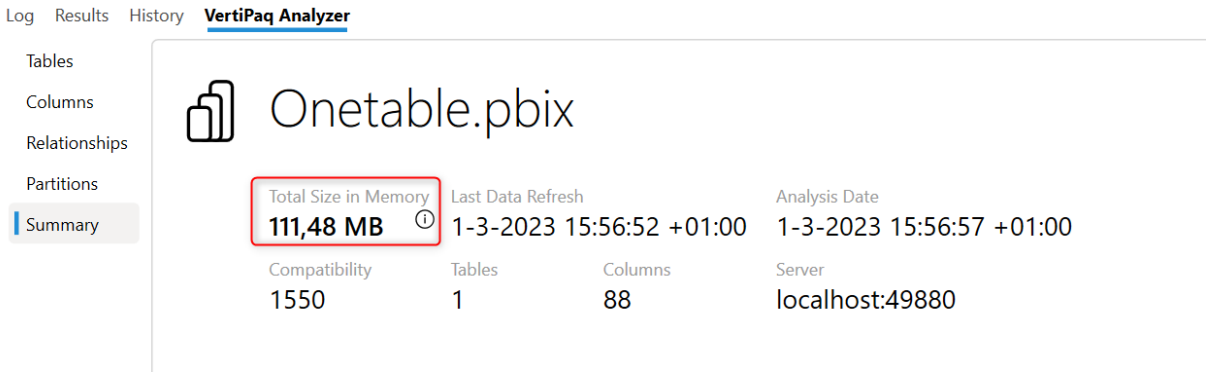
Blok 2

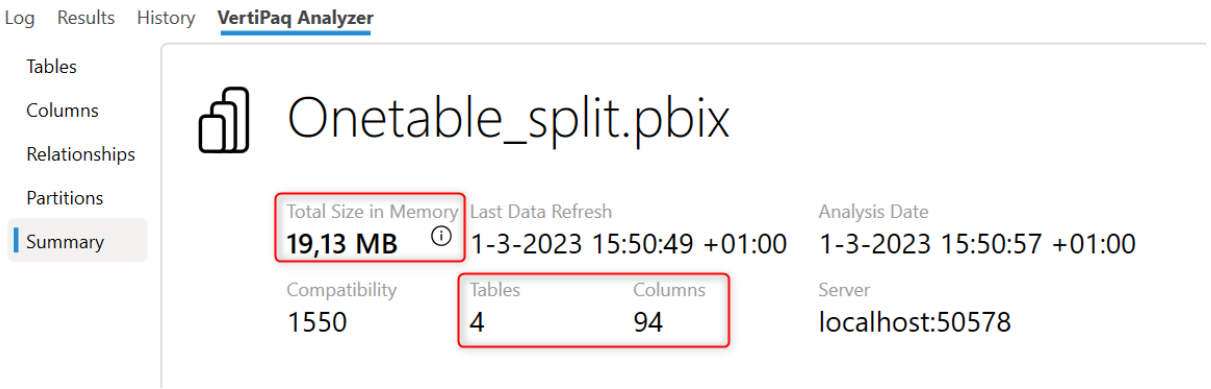
- Nutteloze kolommen verwijderen

Eén brede tabel of een model

Blok 2







dutchBIgeek

Grain kiezen: drie opties, drie prijzen

Blok 2

Probleem Target staat per maand, omzet per dag. Hoe vergelijk je die?

Optie	Resultaat	Engine
Op maand-grain	Correct, geen dagdetail	2 SE-queries, goedkoopst
Toerekenen in DAX	Totaal klopt niet	6 SE-queries, 51% FE
Dag-fact vooraf	Correct, met dagdetail	2 SE-queries, meeste SE CPU

Cardinaliteit

- Eén kolom die je alleen optelt: 55% van de tabel.
- DateKey met 1.095 waarden: 33 MB voor dezelfde rijen.
- 45× verschil, puur door cardinaliteit.

VertiPaq Analyzer

- Beschikbaar op: *www.sqlbi.com/tools/vertipaq-analyzer*



Relationships en DISTINCTCOUNT

Blok 2

Relationships

- De kosten volgen het aantal unieke waarden in de key, niet in de kolom waarop je filtert.
- Vanaf zo'n 100.000 unieke waarden is het meetbaar, boven een miljoen merkbaar trager.
- Bidirectional: loopt een filter van de many-side naar de one-side, dan voert de engine extra subqueries uit.

DISTINCTCOUNT

- Kosten groeien met de cardinaliteit van de kolom.
- Resultaten zijn niet op te tellen. Elke cel in een matrix is een aparte telling.
- Tel waar het kan in een kleinere tabel, of verlaag de cardinaliteit in het model.
- Meet het. Op lage cardinaliteit is het goedkoop.

Relationships gemeten: VertiPaq Analyzer

Blok 2

Relationship (vanuit Sales)	Grootte	Unieke waarden
OrderDateTime → LocalDate	3,2 MB	1.979.159
DeliveryDateTime → LocalDate	3,2 MB	1.978.979
CustomerKey → Customer	790 KB	296.292
ProductKey → Product	4 KB	2.500
Order Date → Date	1,8 KB	1.096
StoreKey → Store	56 bytes	60

Wat je ziet

- Bijna 90% van de 7,1 MB komt van relationships die niemand heeft gemaakt: auto date/time.
- Datetime tot op de seconde maakt die keys bijna 2 miljoen waarden groot.
- Customer kost 200 keer zoveel als Product. De prijs volgt de key.

Many-to-many: bridge met CROSSFILTER

Blok 2

```
Omzet medewerker =
CALCULATE (
    [Sales Amount],
    CROSSFILTER ( Bridge[SalesKey],
        FactSales[SalesKey], BOTH ) )
```

Variant	Tijd
Bidirectional in het model	4,2 s
CROSSFILTER in de measure	4,3 s
Toerekenen met DIVIDE per bridge-rij	13,7 s (callback)

Waarom CROSSFILTER

- Zelfde queryplan, zelfde snelheid. De prijs zit in de key: ±3,4 miljoen unieke SalesKeys.
- Scope: alleen deze measure betaalt, niet elke query die FactSales via employee filtert.
- Zichtbaar: de logica staat in DAX, niet verstopt in een relationship.
- Rijen tellen op tot meer dan het totaal. Bij many-to-many klopt dat.

Oefening blok 2

Blok 2

1

Grootste kolommen

Open VertiPaq Analyzer op Trage-Contoso en noem de drie kolommen met het meeste geheugen. Zijn ze nodig?

2

Kolom verkleinen

Verlaag de cardinaliteit van één kolom in Power Query of de bron en meet het verschil in modelgrootte.

3

Relationships

Zoek de relationship met de grootste key column. Is die cardinaliteit nodig?

dutchBIgeek

Engine en materialisatie

“Waarom is dezelfde query soms twintig keer sneller?”

BLOK 3

In dit blok

- FE en SE
- Materialisatie
- Expanded tables
- Cache
- Fusion
- Auto-exist
- ALLSELECTED

Twee engines

Formula engine (FE)

- Begrijpt alle DAX
- Werkt met één thread
- Krijgt datacaches van de storage engine en rekent verder
- Heeft geen cache

Storage engine (SE, VertiPaq)

- Kent alleen eenvoudige bewerkingen: scannen, filteren, groeperen, optellen
- Werkt met meerdere threads
- Levert tussentabellen (datacaches) aan de formula engine
- Heeft wel een cache

Geheugengebruik van een query

- **Eenvoudige query's vereisen wat geheugen**
- **Complexe query's vereisen meer geheugen:**
  - Spoolen van tijdelijke waarden
  - Materialiseren van datasets
- **Cache vereist geheugen**
- **Materialisatie is het grote probleem**

Wat is materialisatie?

SE

Storage engine scant en groepeert

Snel, met meerdere threads

Cache

Datacache naar de formula engine

Hoe meer rijen, hoe meer geheugen en tijd

FE

Formula engine rekent verder

Eén thread, per rij in de cache

Materialisatie

De storage engine levert een tussentabel die veel groter is dan het resultaat. Een visual met 10 rijen kan een datacache van miljoenen rijen nodig hebben. Die staat ongecomprimeerd in het geheugen en wordt door één thread verwerkt.

Wanneer materialiseert de engine?

- **Materialisatie voor:**
  - Complexe verbindingen
  - Complexe iteratoren
- **Tijdelijke gegevens gespoold voor verdere verwerking**
- **Geheugeneisen:**
  - Misschien meer dan de hele database
  - Gespoolde gegevens worden niet gecomprimeerd

Materialisatie herkennen

Blok 3

In Server Timings en queryplan

- Rows bij een SE-query is veel groter dan het aantal rijen in de visual
- Groot aandeel FE-tijd ten opzichte van SE-tijd
- Veel bijna gelijke SE-queries
- Hoge Records bij een spool in het fysieke plan

Typische oorzaken

- FILTER over een hele tabel als filter argument
- Iteratie over een kolom met hoge cardinaliteit, zoals een ordernummer
- Voorwaarden over kolommen uit verschillende tabellen
- Geneste iterators met context transition

Expanded tables: dimensions reizen mee

Blok 3

Sales (expanded)

Sales		Product		Customer	Date	Store
Quantity	Net Price	Brand	Color	Country	Fiscal Year	Store Name
2	84,07	Litware	Red	Netherlands	2025	Store 09
1	190,74	Contoso	Blue	Germany	2025	Online

Eigen kolommen *Erbij via de relationships (many-to-one), niet fysiek opgeslagen*

Alleen van many naar one. Sales (expanded) bevat Product, maar Product (expanded) bevat geen Sales. Daarom filtert Product wel Sales, en Sales niet Product, tenzij de relationship bidirectional is.

Wat dit betekent

- FILTER (Sales, ...) filtert al deze kolommen tegelijk.
- Een filter op Sales[Quantity] raakt alleen die ene kolom.
- Hoe breder het filter, hoe groter de datacache.

Materialisatie vermijden: complexe condities

Blok 3

- Complexe conditions, waarbij expressions over meerdere kolommen worden gebruikt, zijn gevaarlijk voor materialisatie
- Hoe eerder ze in het proces berekend worden hoe beter

```
bla =
CALCULATE (
    SUMX (
        FILTER (
            FactSales,
            OR (
                FactSales[CurrencyKey] > FactSales[SalesAmount],
                FactSales[channelKey] = 1
            )
        ),
        FactSales[StoreKey] = 77
    )
)
```

Materialisatie voorkomen


```
-- filter op de hele expanded table
Te laat traag =
CALCULATE (
    [Sales Amount],
    FILTER (
        Sales,
        Sales[Delivery Date]
            - Sales[Order Date] > 7 ) )
```

```
-- filter op alleen de nodige kolommen
Te laat snel =
CALCULATE (
    [Sales Amount],
    KEEPFILTERS ( FILTER (
        ALL ( Sales[Order Date],
              Sales[Delivery Date] ),
        Sales[Delivery Date]
            - Sales[Order Date] > 7 ) ) )
```

Traag

Het filter op de tabel dwingt een slechter plan af met grote datacaches, ook al zijn maar een paar kolommen nodig.

Snel

Alleen de unieke combinaties van twee kolommen. KEEPFILTERS laat de slicers intact.

Cache: koud en warm meten

Wat er gecachet wordt

- De SE bewaart datacaches en hergebruikt ze bij een identieke xmSQL-query.
- De FE heeft geen cache. FE-werk wordt elke keer opnieuw gedaan.
- Callbacks worden niet gecachet.
- De cache heeft standaard 512 plaatsen per database.

Hoe je meet

- Koud: Clear Cache aan. Dit is de meting voor optimaliseren.
- Warm: de query twee keer draaien. Dit is wat een gebruiker vaak ervaart.
- Grote verschillen tussen koud en warm betekenen veel SE-werk.
- Weinig verschil met veel FE-tijd betekent: herschrijven helpt meer dan het model.

Fusion: meerdere measures, één SE-query

Blok 3

Twee vormen

- Verticaal: measures met hetzelfde filter maar een andere aggregatie in één query. Omzet en Aantal samen.
- Horizontaal: measures die alleen verschillen in een filter op dezelfde kolom. Rood en Blauw samen, gegroepeerd op kleur.
- Een callback breekt verticale fusion niet. Nulmeting B: vaak één of twee SE-queries.

Wat horizontale fusion blokkeert

- Bereikfilters of filters die tijdens de query berekend worden, zoals een datumbereik
- TREATAS als filter
- DISTINCTCOUNT: fuseert niet, ook niet met een SUM op hetzelfde filter
- Filters met meerdere waarden, of verschillen op meer dan één kolom

Fusion gemeten: nulmeting B

Blok 3

Query	SE-queries	Totaal
Nulmeting B (4 measures)	2	9 ms
Plus # Klanten (DISTINCTCOUNT)	3	17 ms

```
-- 1: vertical fusion
SELECT Product[BrandName],
  SUM ( Sales[SalesQuantity] ), SUM ( @$Expr0 )
-- 2: horizontal fusion
SELECT Product[BrandName], Product[ColorName], ...
WHERE Product[ColorName] IN ( 'Blue', 'Red' )
-- 3: eigen scan, parallel x2,3
SELECT Product[BrandName], DCOUNT ( Sales[CustomerKey] )
```

Wat je ziet

- Omzet en Aantal: één scan, twee aggregaties.
- Rood en Blauw: één scan op kleur, met beide waarden in de WHERE. De FE splitst het daarna.
- DISTINCTCOUNT komt erbij als eigen scan. De andere fusion blijft intact.
- Rows is een schatting: 266, terwijl het er hooguit 28 zijn.

Auto-exist: als twee filters één worden

Blok 3

Probleem Slicers op Product[Brand] en Product[Color]. Een measure haalt het filter op Color weg, maar de uitkomst klopt niet.

Wat er gebeurt

- SUMMARIZECOLUMNS combineert filters op kolommen uit dezelfde tabel tot één filter met alleen bestaande combinaties.
- Haal je Color weg, dan blijft dat gecombineerde filter op Brand staan.
- Merken zonder de gekozen kleur vallen dan ook weg.

Wat je eraan doet

- Weet dat het bestaat; het verklaart veel 'onlogische' totalen.
- Zet onafhankelijke attributen in aparte dimensions. Over tabellen heen gebeurt geen auto-exist.
- Alleen SUMMARIZECOLUMNS doet dit, CALCULATETABLE niet.

ALLSELECTED en shadow filter contexts

Blok 3

```
% van selectie =
DIVIDE (
    [Sales Amount],
    CALCULATE ( [Sales Amount],
        ALLSELECTED ( Product[Brand] ) ) )
```

Werkt goed

Direct in een visual. ALLSELECTED herstelt dan de filters zoals ze buiten de visual stonden.

Wat het echt doet

- Elke iterator maakt een verborgen kopie van de filter context: een shadow filter context.
- ALLSELECTED herstelt de laatste daarvan, niet per se die van de visual.
- Roep je deze measure aan binnen je eigen SUMX, dan verandert de uitkomst.
- Regel: ALLSELECTED alleen in measures die direct in een visual staan.

Oefening blok 3

Blok 3

1

Table filter vervangen

Herschrijf een meegeleverde measure met FILTER over Sales naar een column filter en vergelijk Rows.

2

Fusion breken

Voeg aan de query uit nulmeting B een measure toe die fusion breekt. Hoeveel SE-queries krijg je?

3

ALLSELECTED

Vind de fout in een meegeleverde measure die ALLSELECTED binnen een iterator gebruikt.

dutchBIgeek

DAX Studio en optimaliseren

“Dit rapport doet er twintig seconden over.”

BLOK 4

In dit blok

- Meetopstelling
- Server Timings
- xmSQL en queryplan
- Callbacks
- Herschrijftechnieken

DAX Studio: de meetopstelling

Blok 4

Verbinden

Power BI Desktop open, DAX Studio via External Tools. Zo zit je op hetzelfde model.

Query ophalen

Performance Analyzer, Copy query. Dan meet je precies wat de visual vraagt.

Clear cache

Clear Cache Before Running aan voor een koude meting, uit voor een warme.

Traces aan

Server Timings en Query Plan aanzetten voordat je de query draait.

DEFINE MEASURE

Test een variant in de query zelf, zonder iets in het model te wijzigen.

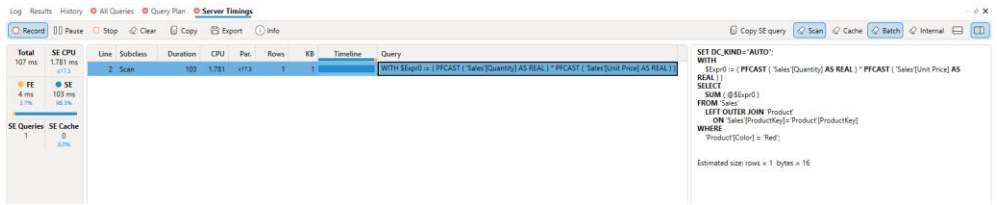
Herhaald meten

Draai elke variant meerdere keren en kijk naar het patroon, niet naar één run.

Server Timings

De **Server Timings** functie in DAX Studio toont gedetailleerde tijdmetingen voor elke fase van een DAX-query, verdeeld over de **Storage Engine** en **Formula Engine**

Dit helpt je om bottlenecks te identificeren en de prestaties van je queries te optimaliseren.



Server Timings lezen

Blok 4

Total	Totale duur van de query in milliseconden
SE CPU	Rekentijd van alle SE-threads samen. Gedeeld door SE-tijd geeft de parallelism
FE	Tijd in de formula engine. Hoog aandeel wijst op grote datacaches. Callbacks tellen mee als SE CPU
SE queries	Aantal xmSQL-queries. Veel bijna gelijke queries wijst op iteratie per rij
Rows / KB	Grootte van elke datacache. Vergelijk met het aantal rijen in de visual
SE cache	Queries uit de cache. Koud bijna nul; hergebruik binnen dezelfde query telt ook mee

Omzet A ontleed: koud en warm

Blok 4

Query	Totaal	FE	SE	SE cache
Omzet A koud	1.077 ms	275 ms	802 ms	1 van 4
Omzet A warm	695 ms	290 ms	405 ms	2 van 4
Omzet C koud	23 ms	3 ms	20 ms	0 van 1
Omzet C warm	3 ms	3 ms	0 ms	1 van 1

```
-- regel 2: 665.620 rijen, 44 MB
WITH $Expr0 := ( CAST ( PFCAST ( Sales[Quantity] ...
-- regel 4: per rij terug naar de FE
WITH $Expr0 := [CallbackDataID ( SUMX ( Sales,
    Sales[Quantity] * Sales[Net Price] ) ) ]
```

Wat je ziet

- Materialisatie: elke verkoopregel van 2025 gaat als datacache naar de FE.
- Callback: de measure wordt per rij via CallbackDataID uitgerekend.
- Warm komen de gewone scans uit de cache. De callbacks draaien opnieuw.
- Warm is Omzet A ruim 200 keer trager dan Omzet C, voor hetzelfde antwoord.

Semi-additive meten: voorraad

Probleem Voorraad mag optellen over product en winkel, niet over tijd.

Measure	Tijd	Server Timings
SUM (fout)	12 ms	1 scan, alleen SE
AVERAGE	25 ms	2 scans
Slotstand, LASTNONBLANK	60 ms	3 scans, leest de fact twee keer, 43% FE

Wat je leert

- De snelste measure is hier fout.
- Correct kost scans en FE-tijd. Meet wat het kost.
- Lees per variant het aantal SE-queries en het FE-aandeel.

xmSQL: wat de storage engine uitvoert

Blok 4

```
WITH
    $Expr0 := ( 'Sales'[Quantity]
                * 'Sales'[Net Price] )
SELECT
    'Product'[Brand],
    SUM ( @$Expr0 )
FROM 'Sales'
    LEFT OUTER JOIN 'Product'
        ON 'Sales'[ProductKey] = 'Product'[ProductKey];
```

Vereenvoudigd voorbeeld van de query achter Omzet per merk.

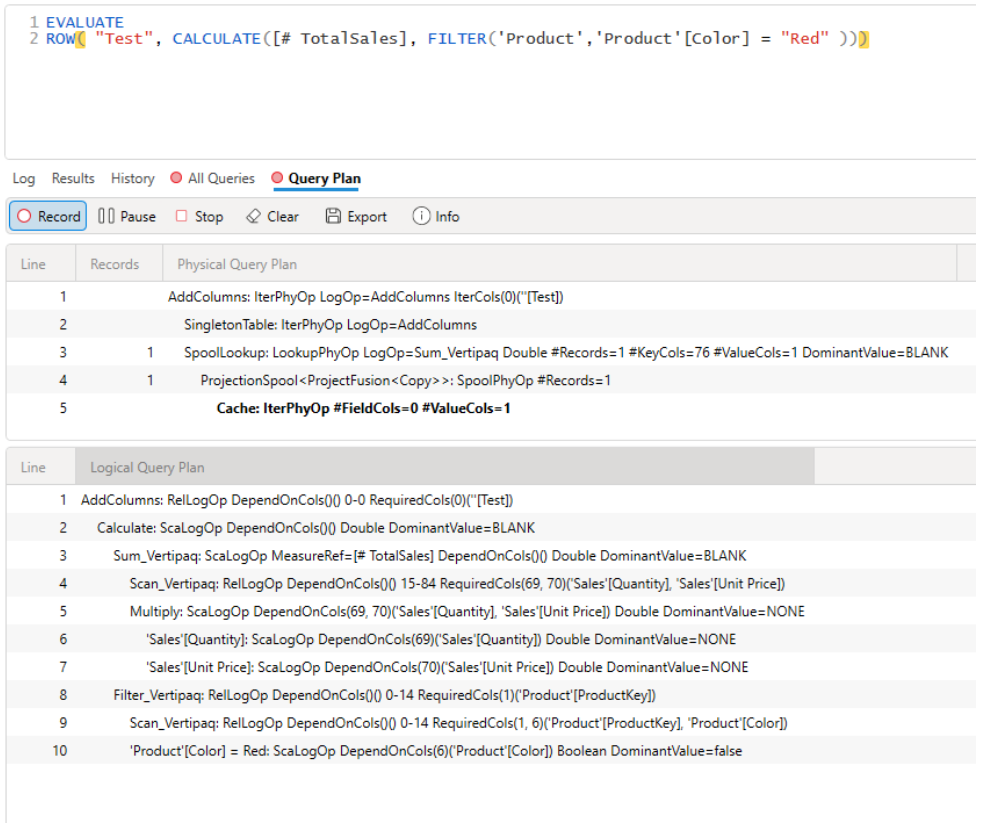
Waar je op let

- Elke join is een relationship uit het model
- SELECT-kolommen bepalen de grootte van de datacache
- CallbackDataID in een expressie betekent een callback
- WHERE met een lange lijst waarden wijst op een groot filter

Queryplan

Blok 4

De **Query Plan** functie in DAX Studio geeft inzicht in hoe DAX-queries worden uitgevoerd door de engine. Het toont een gedetailleerd uitvoeringsplan dat laat zien welke stappen de formule-engine en de opslag-engine nemen om een query te verwerken. Dit omvat informatie over de gebruikte algoritmen en de volgorde van operaties. Door het queryplan te analyseren, kun je identificeren waar mogelijke prestatieproblemen liggen en beter begrijpen hoe je je DAX-code kunt optimaliseren voor snellere en efficiëntere uitvoering.



Een queryplan lezen in vier stappen

Blok 4

1	**Fysiek plan eerst**	Het logische plan zegt wat, het fysieke plan zegt hoe. Voor performance telt het fysieke.
2	**Zoek de grootste Records**	Sorteer of scan op de kolom Records. Daar zit het werk.
3	**Koppel aan Server Timings**	Een spool met veel records hoort bij een SE-query met veel rijen.
4	**Vind het werk per rij**	Operatoren die per rij een berekening doen in de FE zijn de kandidaat om te herschrijven.

Callbacks in de storage engine

Blok 4

```
Premium traag =
SUMX (
    Sales,
    IF ( Sales[Unit Price] >= 500,
         Sales[Quantity] * Sales[Net Price] ) )
```

```
Premium snel =
CALCULATE (
    [Sales Amount],
    KEEPFILTERS ( Sales[Unit Price] >= 500 )
)
```

Wat je ziet

- Kan de storage engine een expressie niet zelf uitrekenen, dan roept hij per rij de formula engine aan.
- In de xmSQL heet dat CallbackDataID. De tijd telt mee als SE CPU.
- Callbacks worden niet gecachet. De communicatie per rij is wat het traag maakt, niet een gebrek aan threads.
- Herschrijf de voorwaarde als filter. Dit is nulmeting C.

Callback gemeten: nulmeting C

Blok 4

Measure	Totaal	SE	FE	Scan
Premium traag	10 ms	7 ms	3 ms	callback
Premium snel	5 ms	3 ms	2 ms	WHERE

```
-- traag: de IF gaat per rij terug naar de FE
WITH $Expr0 := [CallbackDataID ( IF (
    Sales[UnitPrice] >= 500,
    Sales[SalesQuantity] * Sales[UnitPrice] ) ) ]

-- snel: een gewoon filter in de storage engine
WHERE PFCASTCOALESCE ( Sales[UnitPrice] AS INT )
    >= COALESCE ( 5000000 )
```

Wat je ziet

- Beide koud gemeten, SE cache 0. Eén scan per measure.
- Op deze data is het verschil klein. Het patroon telt: een callback per rij of een filter.
- 500 wordt 5000000: Fixed Decimal wordt opgeslagen als geheel getal maal 10.000.
- Groter verschil? Meet Premium op Trage-Contoso, met 2 miljoen rijen.

Callback in de praktijk: levertijd


```
Dagen tot verzending (traag) =
AVERAGEX (
    FactOrders,
    DATEDIFF ( FactOrders[OrderDate],
        FactOrders[ShipDate], DAY ) )
```

```
Dagen tot verzending =
AVERAGE ( FactOrders[DaysToShip] )
```

Variant	Tijd
DATEDIFF per rij	344 ms
Kolom DaysToShip	20 ms

Wat je ziet

- VertiPaq kent alleen de vier basisbewerkingen, eenvoudige aggregaties en joins. DATEDIFF valt erbuiten: per order een CallbackDataID.
- Zelfde antwoord, ongeveer 17 keer trager.
- Met de kolom in het model doet VertiPaq alleen een som en een telling.
- Model eerst: reken vast wat per rij vastligt.

Herschrijftechnieken

Blok 4

Kolom, geen tabel

Filter op de kolommen die je nodig hebt, niet op een hele tabel.

Voorwaarde als filter

Zet een IF in een iterator om naar een filter in CALCULATE.

Kleinste tabel

Itereer over een dimension of VALUES van een kolom, niet over de fact table.

Niet op fusion leunen

Eén goed geschreven filter is sneller dan meerdere measures die fusion samenvoegt.

Variabelen

Bereken één keer. Let op: een VAR verandert niet door een latere CALCULATE.

Model eerst

Lost een kolom in de bron of een plattere dimension het op? Dan is dat beter dan DAX.

Oefening blok 4

Blok 4

1

Koud en warm meten

Meet een trage visual koud en warm. Noteer totaal, SE- en FE-tijd en de grootste datacache.

2

Callback opsporen

Vind de callback in een meegeleverde measure, herschrijf hem en meet opnieuw.

3

Queryplan

Vergelijk het fysieke plan van een measure voor en na herschrijven. Waar zakken de records?

dutchBIgeek

Praktijkcase: een traag rapport

“Deze pagina moet binnen twee seconden laden.”

PRAKTIJKCASE

In dit blok

- Rapport meten
- Oorzaak vinden
- Herschrijven en meten
- Presenteren

De case

Praktijkcase

Probleem Een salesrapport per merk en boekweek laadt in meer dan tien seconden.

Op de pagina

- Matrix: merk per boekweek
- Omzet, omzet vorig jaar en groei %
- Omzet van te late leveringen, % van selectie
- Slicers op jaar, kleur, merk en land

Opdracht

- Meet de pagina en vind de traagste visual
- Zoek de oorzaak: model, fusion, materialisatie, callback of transitie
- Herschrijf en meet opnieuw, koud en warm
- Presenteer in twee minuten: oorzaak, oplossing, winst

Case-measure: vorig jaar per boekweek

Praktijkcase

```
Omzet VJ (boekweek) =
VAR HuidigJaar = MAX ( 'Date'[Fiscal Year] )
RETURN
    CALCULATE (
        [Sales Amount],
        REMOVEFILTERS ( 'Date' ),
        'Date'[Fiscal Year] = HuidigJaar - 1,
        VALUES ( 'Date'[Fiscal Week] )
    )
```

```
Groei % =
DIVIDE ( [Sales Amount] - [Omzet VJ (boekweek)],
         [Omzet VJ (boekweek)] )
```

Wat je terugziet

- Blok 1: VALUES wordt berekend in de oorspronkelijke context en toegepast na REMOVEFILTERS.
- Blok 1: alleen column filters.
- Blok 2: controleer of deze measure fusion met Omzet breekt.

Nameting: dezelfde drie voorspellingen

Afsluiting

A

KEEPFILTERS en ALL in één CALCULATE

Wat staat er op de rij Blue, en welke stap verklaart het?

B

Vier measures in één query

Hoeveel SE-queries, en wat zou fusion breken?

C

IF in een iterator of een filter

Welke is sneller, en waar zie je dat?

Elke keuze heeft een prijs

Afsluiting

Grain eerst

De grain bepaalt het type fact, en het type fact bepaalt je DAX.

Drie soorten prijs

Snelheid, geheugen of correctheid. Een keuze kost altijd één van de drie.

De prijs verschuift

Hij verdwijnt nooit. Kies bewust en meet wat je betaalt.

Snel is niet correct

De engine waarschuwt je niet. Controleer de uitkomst, niet alleen de tijd.

dutchBIgeek

Heb je vragen of interesse? Neem contact op.



Peter van den Bos

Business Intelligence Consultant

peter@dutchbigeek.nl

+31 6 13760795

dutchbigeek.nl